COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	10/12/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

Code:

The code implements an adjacency list to show relationships of each edge in the graph

Class DiaGraph:

```
adjNode* getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // point new node to current head
    return newNode;
}
```

Inside the class DiaGraph, we have the getAdjListNode, which adds new nodes into the adjacency list

```
public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
    for (int i = 0; i < N; i++)
        delete[] head[i];
    delete[] head;
    }
```

In the public part of the DiaGraph class, we create a constructor to allocate the new node, initialize the head

pointer by pointing to null, constructing a directed graph by adding edges, and deallocate the memory using a destructor class to avoid having a garbage address for the head.

```
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
```

The display_AdjList function displays all the adjacent vertices of the given vertex show the relationship of each edges to the other edges in the graph.

Main/Driver Code:

```
int main() {
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list " <<std::endl <<(edges, n,
N) <<std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
        }
    return 0;
}
```

The main code we have edges with three numbers. The first number is the start edge, the second number represents the edge where the start edge is pointing to, and the third number represents the weight. Then the edges are displayed using the display_AdjList function implement inside a for loop.

Console/Output:

```
Graph adjacency list
6
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

The output shows the relationship of edge in a directed graph. In the output the relationship of each edge as it

shows:
0 is connected to 1 with a weight of 2
0 is connected to 2 with a weight of 4
1 is connected to 4 with a weight of 3
2 is connected to 3 with a weight of 2
3 is connected to 1 with a weight of 4
4 is connected to 3 with a weight of 3
5 is connected to null

Table 6.1: Adjacency List Implementation

Code:

This code demonstrates how depth-first search (DFS) works to show which graph to traverse first.

Struct Edge:

```
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
    return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
    return this->weight > e.weight;
    }
};
```

In the structure Edge, this is where all the information of the edge is stored form. It stores the source edge, the destination edge, and the weight of it, and the operators compares the two edges based on their weights.

std::ostream &operator

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

This function tells the program how to output the whole graph in the console. We can see that by looking at the function, which has std::ostream that is used for outputting data, and the <<, commonly used when using std::cout.

Class Graph:

```
template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }
    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }
    void add_edge(Edge<T> &&e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a strea
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

The Graph class is the one of the most important parts of the code as it contains all the function that is need to create a graph, in this class, the public contains the following:

Graph (size_t N) – constructs a graph (constructor)
vertices() – returns the amount of vertices in the graph
edges() – provides access to the edges
add_edge () – adds an edge
outgoing_edges () – retrieves the edges

While the private contains the following parameters:
size_t V – Stores the number of vertices the graph allows
std::vector<Edge<T>> edge_list – stores the edges as a vector

Depth-First Search function:

This function is the most important part of the program because it does a depth-first search to the graph.

```
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

In this program, the following keeps track of which tracks which:
stack – Holds the vertices
visit_order – Records the vertices that have been visited
visited – Keeps track of the vertices that are already visited

The first vertex to be pushed is the starting vertex (which in this case is 1), and while the stack is not empty, it will continue to get the next vertex and determine if the current_vertex is visited or not. If the vertex is still not visited, it will mark that vertex as visited and stores is at the visit_orde, find its neighbors and returns the visit_order that can be outputted.

Reference Graph:

```
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Creates a a reference graph to be used for the DFS implementation. It also adds the each of the edges to the graph.

Test DFS and Main/Driver Code:

```
template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
```

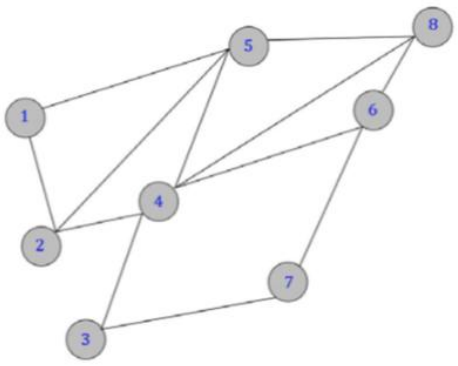
The test DFS uses the reference graph to implement the search and outputs the result.

Console/Output:

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

The outputs show not only the test DFS to the reference graph, but also shows the adjacency list of each of the edges.



The graph above shows how the graph looks like when visualize. In the output DFS chose the path:
1 -> 5 -> 8 -> 6 -> 7 -> 3 -> 4 -> 2

As the path to traverse the whole graph while visiting it exactly once as DFS is randomized

Table 6.2: Depth-First Search Implementation

Code:

This code shows the BFS code algorithm, which is similar to DFS, which some fixes from the code.

```
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};
```

This code is very similar to the DFS, so all of mentioned things will be similar, but there are some tweaks that makes this BFS, instead of another DFS.

The following parts of code are the same as DFS traversal

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

std::ostream &operator

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}
    auto vertices() const {
        return V;
    }
    auto &edges() const {
        return edge_list;
    }
    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

class Graph

```cpp
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

Reference Graph

```cpp
template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

test BFS and Main/Driver Code

Majority of the code are very similar and to not need explaining, except for one and thos this the:

Breadth-First Search

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

Despite being a minor tweak, it still is importance to point out because BFS commonly uses queue, instead of a stack to search each node. The rest of the code have are just the same as the DFS, but stack is changed as queue.

Output:

Using the same graph, this is the output of BPS

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

The BFS traversal is as follows:
1 -> 2 -> 5 -> 4 -> 8 -> 3 -> 6 -> 7

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
To accomplish the task, the user must use DFS to visit the map. In the problem, it shows how after choosing a location, it want to visit every vertex of the current path until there are no more unvisited vertex in the path, then backtracks to go to the other vertex and explore that path, eventually getting deeper in the graph, until there more vertex unvisited and no more backtracking to an unvisited node.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

All of the traversal strategies for trees are equivalent to DFS, but the closest of them all is the Pre-order Traversal. Pre-Order traversal is a tree traversal that starts at root and traverses to the left most node. This is similar to DFS because its traversal starts at a chosen node, traverse deeper on one side until there is nowhere to go, and traverses the other side.

To show the similarity of the Pore-Order to DFS traversal, I made graph showing know they are very similar to each other with the similar structure and node labels.

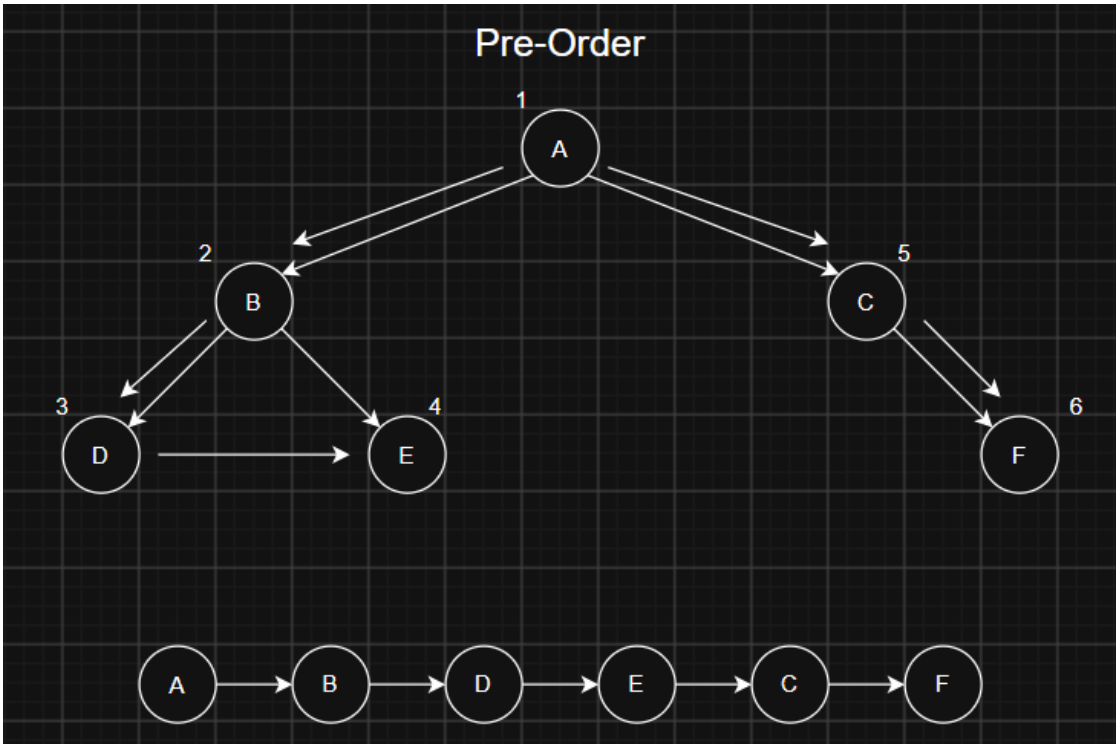


Image 1: Pre-Order Traversal

In this traversal, it visits the root, then traverses the left side of the tree, then jumps to the right side after.

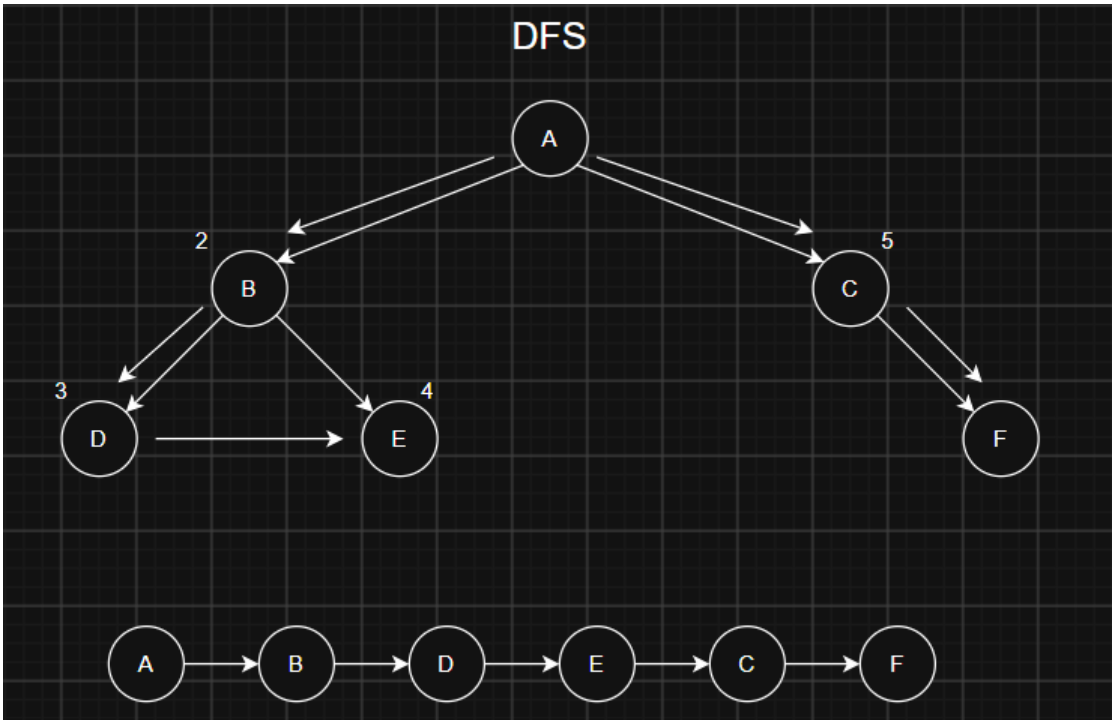


Image 2: DFS Traversal

In this one with a similar structure It starts at A, then traverses the left most side and goes deeper, once it can't go

deeper, it jumps and traverses the right side. This shows how similar DFS is to Pre-Order Traversal.

To further their similarity here is the pseudocode and code implementation of both the DFS and Binary Search

```
DFS(Graph G, t_size dest):
    Create a stack
    Create a set of visited vertices
    Create visit_order list

    Push the starting vertex onto the stack

    WHILE the stack is not empty:
        current = TOP of stack
        POP the stack

        IF current is not in visited:
            Add current to visited
            Add current to visit_order

            FOR each neighbor of current:
                IF neighbor is not in visited:
                    PUSH neighbor onto the stack

    RETURN visit_order
```

Image 3: DFS Pseudocode

```
PREORDER():
    IF left is not NULL:
        PREORDER(node.left)

    IF right is not NULL:
    PREORDER(node.right)
```

Image 4: Pre-order Pseudocode

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Image 5: DFS Traversal Code Implementation

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout <<key <<" ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}
```

Image 6: Pre-Order Code Implementation

Despite DFS not having a left or right, it pushes the starting vertex (or the root in trees) to the stack and inserts it to the node, and every neighbor it passes to, it records the traversed node, and after every node is traversed, the visited_node is returned and will outputted.

3. In the performed code, what data structure is used to implement the Breadth First Search?
In the performed code, the data structure used to perform BFS is the queue. In BFS, it processes the first node that was traversed, which basically follows the FIFO (First In, First Out) method, and the in the code, the queue is used to make an empty line where all the traversed elements will be enqueued or dequeued.

4. How many times can a node be visited in the BFS?
The node in BFS can only be visited once because of the part the code implemented that prevents it from showing up again (like in DFS). In the code there is a parameter called visited, that marks the node that is already traversal as already visited, meaning it can never output or traverse the code twice.

Conclusion:

In conclusion, graphs are a type of data structure that has nodes and edges connecting to other nodes to show the relationship between them. The structure is similar to a tree, but trees have a root, while a graph has a set of starting points before running the code.

The procedure shows the basic creation of a graph with an adjacency matrix, and how to implement BFS and DFS. While the creation of a graph with an adjacency list and DFS traversal were already given, the BFS did have errors in the code itself, but it was easily solved because most of the errors were already revealed in the DFS traversal.

The supplementary featured questions about graphs and which tree traversal closely represents DFS, so the supplementary wasn't too hard. Still, I did have to make a diagram and pseudocode to show the similarity of DFS to Pre-Order, but it wasn't much of a big issue.

Overall, I did pretty well with this Hands-On Activity and felt pretty good, but it was already late since I wasn't able to do it on the other days due to other problems and time constraints. I felt like there was nothing to improve, but I would still say my analysis of DFS and BFS would need a bit of improvement, even though I think I did decent on it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Supplementary Assistance, Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used ChatGPT to assist in answering the questions, but still made the diagrams, pseudocode, and code as my own work.**
Using Grammarly for polish and fixing mistakes made in the code

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.